DocPilot

A Copilot-Inspired Multi-Model Document Intelligence Platform

Task 3: Architectural Tactics and Patterns

S26CS6.401 — Software Engineering

Team No. 38

Domain: Productivity

April 20, 2026

Contents

1 Task 3: Architectural Tactics and Patterns **2**
1.1 3.1 Architectural Tactics 2
1.1.1 Tactic 1 — Passive Redundancy / Spare (Availability) 2
1.1.2 Tactic 2 — Manage Event Rate / Limit Access (Security & Availability) 3
1.1.3 Tactic 3 — Dynamic Model Selection / Introduce Concurrency (Performance) 3
1.1.4 Tactic 4 — Separate Interface from Implementation / Specialize Access (Testability) 4
1.1.5 Tactic 5 — Cancel / Undo + Support User Initiative (Usability) . 5
1.2 3.2 Implementation Patterns 6
1.2.1 Pattern 1 — Strategy Pattern (Agent Mode Orchestration) 6
1.2.2 Pattern 2 — Chain of Responsibility (LLM Request Pipeline) . . 7
1.2.3 Companion Pattern — Command + Memento (Reversible Edits) 9
1.3 3.3 Tactics-to-NFR Mapping Summary 10

Task 3: Architectural Tactics and Patterns

Architectural tactics are design decisions that directly influence a system's ability to achieve a desired quality attribute response. As defined in course literature, a tactic is a characterization of an architectural decision targeted at a specific quality concern. For DocPilot, the tactics below were selected to directly address the NFRs established in Task 1 and the constraints arising from a multi-model, RAG-based document intelligence pipeline.

3.1 Architectural Tactics

DocPilot employs five architectural tactics, each mapped to one or more non-functional requirements. Each tactic is described using the quality-scenario structure (Source, Stimulus, Artifact, Environment, Response, Response Measure) consistent with the Len Bass quality-scenario model.

Tactic 1 — Passive Redundancy / Spare (Availability)

Quality Attribute: Availability

NFR Addressed: NFR6 — Model fallback must initiate within 2 seconds.

Description. When a user submits a query and the primary LLM provider (e.g. GPT-4o) is unavailable—either due to a service outage, rate-limit exhaustion, or network timeout—the system automatically switches to a pre-configured fallback model (e.g. Claude 3 or Gemini 1.5) without requiring any user intervention. This is realized through the *Chain of Responsibility* pattern in the `ModelGateway`, where each handler in the chain attempts to fulfill the request and passes it downstream on failure.

The tactic falls under the **Fault Recovery** branch of Availability Tactics (specifically, *Spare/Passive Redundancy*): a set of backup models is kept warm and configured, ready to receive traffic if the primary fails.

Quality Scenario.

Element	Value
Source	External (LLM provider infrastructure)
Stimulus	Primary model returns a timeout or 503 error (omission fault)
Artifact	`ModelGateway` — Chain of Responsibility handler chain
Environment	Normal operation (user-initiated Q&A or editing request)
Response	Gateway catches the exception, logs the failure, and routes the request to the next provider in the chain
Response Measure	Fallback initiates within **2 seconds**; end-to-end response degradation $\leq$ 20% of normal latency SLA

How it addresses the NFR. The Chain of Responsibility maintains an ordered list of providers. On failure, the next handler is invoked synchronously, bounding the fallback latency by the chain traversal time (negligible in-process) plus the secondary model's time-to-first-token.

Tactic 2 — Manage Event Rate / Limit Access (Security & Availability)

Quality Attribute: Security + Availability

NFR Addressed: NFR1 (Response Latency $\leq$ 10 s), NFR5 (Context Budget $\leq$ 90%).

Description. DocPilot enforces per-user and per-session rate limits on LLM API calls—analogous to HTTP 429 semantics—inside the `ModelGateway`. When a session exceeds its token-per-minute or requests-per-minute quota, subsequent calls are queued or rejected with a graceful error, preventing runaway cost, protecting against prompt-injection abuse, and maintaining fair resource sharing across concurrent users.

From the **Security Tactics** tree, this is a *Limit Exposure / Limit Access* measure: it constrains the volume and nature of interactions a single actor can have with the LLM subsystem. From the **Performance Tactics** tree, it falls under *Manage Event Rate* (Resource Demand reduction).

Quality Scenario.

Element	Value
Source	Internal / External (user session or automated script)
Stimulus	Session submits requests exceeding the defined token-per-minute threshold
Artifact	`ModelGateway` — rate-limit middleware layer
Environment	Normal or overload mode (multiple concurrent users)
Response	Gateway blocks or queues excess requests; returns a structured 429-style response with retry-after metadata
Response Measure	No single session consumes > 25% of total token budget per minute; system latency remains $\leq$ 10 s for compliant sessions

How it addresses the NFR. Rate limiting prevents context-budget exhaustion (NFR5) by bounding the total tokens dispatched per time window, and guards latency SLAs (NFR1) by preventing resource starvation for well-behaved sessions.

Tactic 3 — Dynamic Model Selection / Introduce Concurrency (Performance)

Quality Attribute: Performance

NFR Addressed: NFR1 (Q&A $\leq$ 10 s; diff generation $\leq$ 5 s), NFR5 (Context Budget).

Description. When the user has not specified a preferred model, the system dynamically selects the optimal LLM for the task based on a routing policy: fast/cheap models (e.g. GPT-3.5 Turbo or Haiku) for short inline suggestions, large-context models (e.g. Gemini 1.5 Pro) for cross-document synthesis, and vision-capable models for PDF chart analysis. This is realized via the *Strategy* pattern in the `ModelGateway`, where the routing `ModelSelectionStrategy` is injected at runtime.

From the **Performance Tactics** tree, this is a combination of *Increase Computational Efficiency* (Resource Demand) and *Scheduling Policy* (Resource Arbitration): the gateway selects the least-costly model that satisfies the task's quality requirements, reducing average latency and token consumption.

Quality Scenario.

Element	Value
Source	Internal (Agent Orchestrator dispatching a task)
Stimulus	A Q&A task arrives with no explicit model preference; sporadic event pattern
Artifact	`ModelGateway` — `ModelSelectionStrategy`
Environment	Normal operation
Response	Gateway evaluates task metadata (type, token estimate, priority), selects the optimal provider, and dispatches the request
Response Measure	Q&A response latency $\leq$ 10 s; token cost reduced by $\geq$ 30% compared to always routing to the largest model

How it addresses the NFR. By routing lightweight tasks to faster, cheaper models, the tactic keeps latency well within the 10-second ceiling for routine Q&A while reserving large-context models for the tasks that genuinely need them.

Tactic 4 — Separate Interface from Implementation / Specialize Access (Testability)

Quality Attribute: Testability

NFR Addressed: Developer-facing: ability to achieve $\geq$ 80% pipeline test coverage.

Description. DocPilot exposes a set of *specialized internal endpoints*—accessible only via a test API key or when the service is started in `TEST_MODE`—that allow testers to inject documents directly into any pipeline stage (e.g. bypass ingestion and inject pre-parsed chunks directly into the retrieval stage), inspect intermediate state (chunk embeddings, retrieved context, budget usage), and simulate LLM responses without making real API calls (mock provider injection).

From the **Testability Tactics** tree, this maps to:

- *Separate Interface from Implementation* (Manage I/O): pipeline stages expose clean interfaces that can be driven independently.

- *Specialize Access* (Manage I/O): dedicated test endpoints provide deep observability not available in production mode.
- *Built-in Monitors* (Internal Monitoring): in `TEST_MODE`, each pipeline stage emits detailed trace events.

Quality Scenario.

Element	Value
Source	System tester (QA engineer or CI pipeline)
Stimulus	Integration milestone completed; regression suite triggered
Artifact	DocPilot backend pipeline (FastAPI service in `TEST_MODE`)
Environment	Development / CI environment
Response	All pipeline stages accessible via test endpoints; intermediate state observable; mock LLM responses injectable
Response Measure	Regression suite completes within **5 hours**; pipeline stage coverage $\geq 80\%$

How it addresses the NFR. By decoupling each pipeline stage behind clean interfaces, testers can exercise any stage in isolation without requiring the entire system to be live, dramatically reducing test setup time and enabling high fault-discovery probability.

Tactic 5 — Cancel / Undo + Support User Initiative (Usability)

Quality Attribute: Usability

NFR Addressed: User error recovery; reduction of cognitive overhead when managing document attachments and AI edits.

Description. DocPilot supports two complementary usability tactics:

1. **Undo/Redo for AI Edits:** Every AI-proposed change is encapsulated as a *Command* object. Users can undo any accepted edit and redo it, with the document state managed by the *Memento* pattern. The previous document state is restored within one second.
2. **Confirmation Prompt for Attachment Removal:** When a user removes an attached document from the workspace, the system proactively prompts for confirmation ("Remove *report.pdf* from the workspace? This will clear its indexed context.") before discarding the embedding index for that document. This prevents accidental loss of retrieval context.

From the **Usability Tactics** tree (Runtime branch):

- *Cancel / Undo* (Support User Initiative): direct mapping to the undo/redo feature.
- *Support System Initiative* (user, system, task models): the confirmation dialog is a system-initiated intervention that anticipates user error.

Quality Scenario.

Element	Value
Source	End user
Stimulus	(a) User accepts an AI edit and immediately regrets it; (b) User clicks "Remove" on an attached document
Artifact	DocPilot document editor + Command/Memento stack; document workspace attachment manager
Environment	Runtime
Response	(a) Previous document state restored via undo; (b) Confirmation dialog shown before context is purged
Response Measure	(a) Previous state restored within **1 second**; (b) Accidental removal rate reduced to $< 2\%$ of all removal actions

How it addresses the NFR. The Command + Memento stack provides a complete and bounded undo history, while the system-prompted confirmation before destructive operations prevents unrecoverable loss of retrieval context—a critical concern when document corpora have been indexed over time.

3.2 Implementation Patterns

The submission guidelines require two design patterns with explanations of their roles within the architecture, supported by diagrams where applicable. Three patterns are directly instantiated in DocPilot: *Strategy*, *Chain of Responsibility*, and *Command + Memento*. The two primary patterns are described in detail below; the Command pattern is described as a companion to the Usability tactic.

Pattern 1 — Strategy Pattern (Agent Mode Orchestration)

Role in Architecture. The Strategy pattern is used in the **Agent Orchestration Subsystem** to support multiple distinct agent modes without modifying the orchestrator's core logic. Each agent type—Q&A Agent, Edit Agent, Generate Agent, and Multi-Document Agent—represents a concrete strategy that implements a shared `AgentStrategy` interface. The orchestrator selects and injects the appropriate strategy at runtime based on the resolved user intent.

Problem Solved. Without the Strategy pattern, the orchestrator would require a large conditional block (if-else or switch) to dispatch to the correct agent, making it brittle when new agent types are added. The pattern enables the system to be *open for extension* (new agent modes) and *closed for modification* (orchestrator logic unchanged).

Structural Description.

- **Interface:** `AgentStrategy` — declares `execute(context: AgentContext): AgentResult`
- **Concrete Strategies:**

  - `QAAgentStrategy` — retrieves chunks, budgets context, generates grounded answer
  - `EditAgentStrategy` — generates structured diff, wraps in Command object
  - `GenerateAgentStrategy` — produces attributed draft from corpus
  - `MultiDocAgentStrategy` — aggregates context across multiple document indices
- **Context:** `AgentOrchestrator` — holds a reference to the current strategy; delegates `execute()` to it

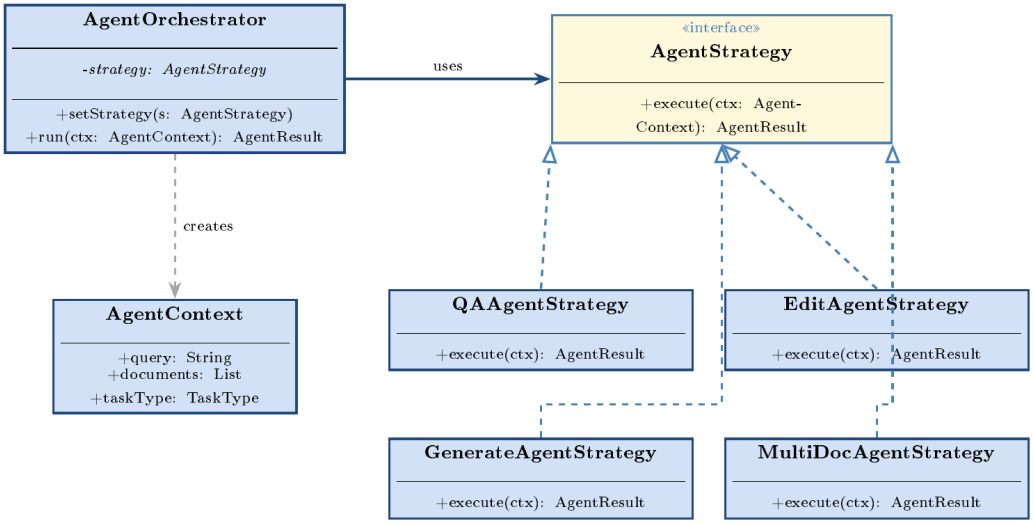

Figure 1: UML Class Diagram — Strategy Pattern (Agent Orchestration)

NFRs Addressed.

- **Modifiability** — new agent modes (e.g. a summarization agent) can be added without touching the orchestrator.
- **Performance** (NFR1) — each strategy is optimized for its task class; the Q&A strategy uses fast retrieval paths while the multi-document strategy uses batched retrieval.
- **Testability** — each strategy can be unit-tested in isolation by injecting a mock `AgentContext`.

Pattern 2 — Chain of Responsibility (LLM Request Pipeline)

Role in Architecture. The Chain of Responsibility pattern is instantiated in *two* places in DocPilot:

1. **Model Fallback Chain (ModelGateway):** Handlers represent LLM providers in priority order. If the primary provider fails, the request propagates to the next handler until it is serviced or the chain is exhausted. This realizes Tactic 1 (Availability).
2. **RAG Request Processing Pipeline:** The sequence of pipeline stages—Intent Resolution → Retrieval → Context Budgeting → Prompt Construction → LLM Call → Citation Grounding → Render—is modeled as a chain where each handler

performs its transformation and passes the enriched context object to the next stage.

Problem Solved. Without this pattern, the pipeline would be implemented as a monolithic function with deeply nested logic, making it difficult to add, remove, or re-order stages (e.g. inserting a re-ranking step between retrieval and context budgeting). The Chain of Responsibility makes each stage independently replaceable and allows the pipeline to short-circuit (e.g. return a cached response before reaching the LLM call stage).

Structural Description — Model Fallback Chain.

- **Abstract Handler:** `ModelProvider` — declares `handle(request): Response`; holds a reference to `nextProvider`
- **Concrete Handlers:**
  - `OpenAIProvider` (primary)
  - `AnthropicProvider` (first fallback)
  - `GeminiProvider` (second fallback)
  - `FallbackErrorHandler` (terminal handler; raises structured error)
- **Client:** `ModelGateway` — constructs and initiates the chain

Structural Description — RAG Pipeline.

- Each pipeline stage implements `PipelineHandler` with `process(ctx: PipelineContext): PipelineContext`
- Stages in order: `IntentResolver` → `Retriever` → `ContextBudgeter` → `PromptBuilder` → `LLMCaller` → `CitationGrounder` → `ResponseRenderer`
- Any stage may short-circuit by returning a completed `PipelineContext` without calling `next`

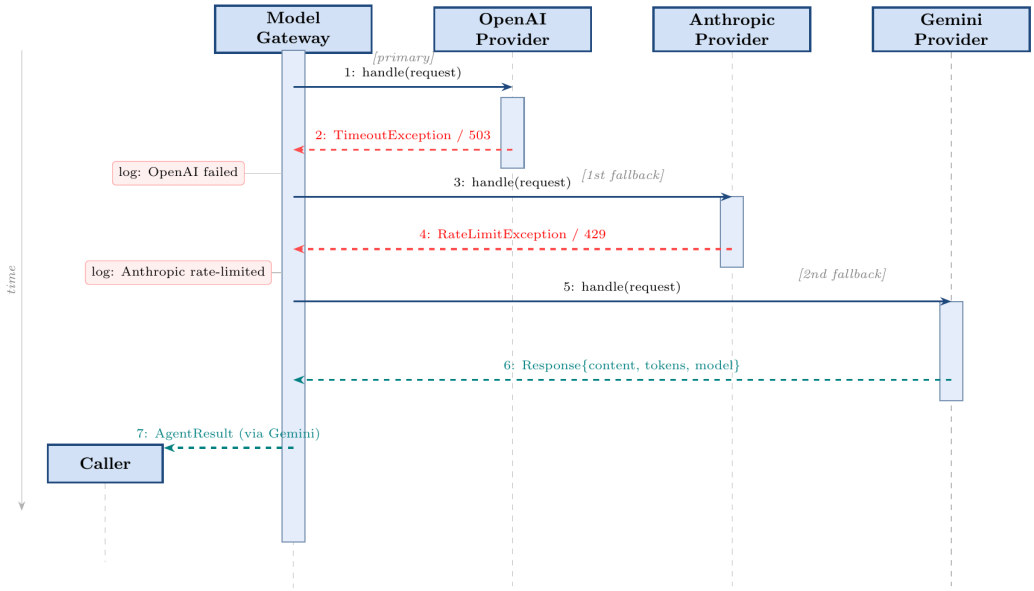

Figure 2: UML Sequence Diagram — Model Fallback Chain (Chain of Responsibility)

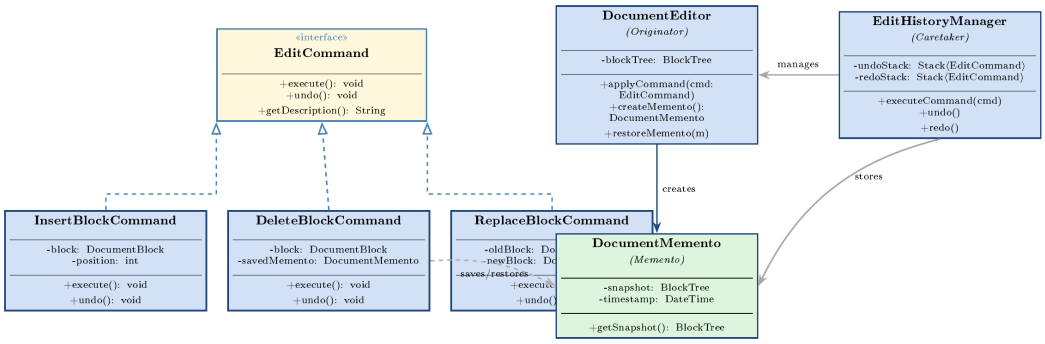

Figure 3: UML Class Diagram — Command + Memento Pattern (Reversible Edits)

NFRs Addressed.

- **Availability** (NFR6) — fallback chain ensures request is serviced even when primary or secondary providers fail.
- **Modifiability** — new providers or pipeline stages can be inserted without modifying existing handlers.
- **Response Latency** (NFR1) — the pipeline can short-circuit early (e.g. return a cached answer from the `Retriever` stage) reducing unnecessary LLM calls.

Companion Pattern — Command + Memento (Reversible Edits)

Role in Architecture. Every AI-proposed edit is encapsulated as a `EditCommand` object (Command pattern) that carries both the `execute()` and `undo()` operations. Before execution, the current document state is saved as a `DocumentMemento` (Memento pattern). The `EditHistoryManager` maintains an ordered stack of commands and mementos, enabling multi-level undo and redo.

Structural Description.

- **Command Interface:** `EditCommand` — `execute()`, `undo()`
- **Concrete Commands:** `InsertBlockCommand`, `DeleteBlockCommand`, `ReplaceBlockCommand`
- **Memento:** `DocumentMemento` — immutable snapshot of the document block tree at a point in time
- **Originator:** `DocumentEditor` — creates and restores mementos
- **Caretaker:** `EditHistoryManager` — manages the undo/redo stacks

NFRs Addressed.

- **Usability** — full multi-level undo/redo with state restoration within 1 second.
- **Availability** — reversibility prevents data loss from incorrect AI edits without requiring a full document reload.
- **Testability** — each `EditCommand` can be unit-tested in isolation against a mock document state.

3.3 Tactics-to-NFR Mapping Summary

Tactic	Quality Attribute	NFR	Pattern Realization
Passive Redundancy / Spare	Availability	NFR6	Chain of Responsibility (ModelGateway)
Limit Access / Manage Event Rate	Security + Availability	NFR1, NFR5	Rate-limit middleware in ModelGateway
Dynamic Model Selection	Performance	NFR1, NFR5	Strategy (ModelSelectionStrategy)
Separate Interface / Specialize Access	Testability	Pipeline coverage	Interface segregation + TEST_MODE endpoints
Cancel/Undo + System Confirmation	Usability	Error recovery	Command + Memento